

Module 10 — Malformed Inputs Supplement

A10 Mishandling of Exceptional Conditions | ZAP-Based Exercise

BBC Academy / JBI Training | Uses: OWASP Juice Shop, ZAP Proxy, Firefox DevTools

Juice Shop connection: when Juice Shop receives requests it cannot parse or validate, it surfaces far more than a production application should — Node.js internals, Sequelize ORM errors, file paths, and framework versions. This exercise uses ZAP to send deliberately malformed requests and trains delegates to recognise and categorise what each response discloses. The same principle is demonstrated against `vulnerable_shop.py` (verbose) and `vulnerable_shop_fixed.py` (hardened) so delegates see the fix in action.

Learning Objectives

By the end of this exercise delegates will be able to:

- Identify at least five categories of malformed request that trigger unhandled errors in Juice Shop.
- Read a ZAP HTTP response and classify exactly what category of information has been disclosed.
- Explain why information disclosure in error responses is a force-multiplier for other OWASP Top 10 attacks.
- Apply the three-layer fix: input validation → custom error handler → structured logging.

Stage 1 — Type Confusion Attacks on REST Endpoints

Send wrong JSON types to `/rest/user/login` — observe what Juice Shop discloses

What to Do	What You Should Observe
1a. Configure ZAP proxy in Firefox Ensure Firefox is pointing to ZAP at 127.0.0.1:8080. Confirm ZAP is recording traffic by visiting any page in Juice Shop. 1b. Open ZAP Requester (Manual Request) Tools → Manual Request Editor. Set method to POST, URL to: <code>http://localhost:3000/rest/user/login</code> Set Content-Type header to <code>application/json</code>. 1c. Send each of the following request bodies in turn. Record the HTTP status code and key phrases from each response body.	Expected responses and disclosures: Request A — null values: <code>{"email": null, "password": true}</code> → 500 Internal Server Error. Response body may contain a Sequelize TypeError and the Node.js call stack revealing internal file paths. Request B — array types: <code>{"email": [], "password": {}}</code> → 500 or 400. ORM validation failure message may name the column and expected type. Request C — integer email: <code>{"email": 12345, "password": [1,2,3]}</code> → Note whether the response references the framework version (Express/Sequelize). Record any file path disclosed.
1d. For each response, highlight and record: • The HTTP status code • Any framework or library name disclosed • Any internal file path disclosed • Any database type or table/column name disclosed • Any version string disclosed	What each disclosure enables: Framework name + version → Attacker searches NVD/Exploit-DB for known CVEs against that exact version. Internal file path → Confirms server OS, directory structure, and potentially the app root for path traversal attempts (Stage 2). Column/table name → Attacker now knows the exact schema for SQL Injection payloads — no blind guessing required.

Stage 2 — Path Traversal and Non-Existent Route Probing

Probe Juice Shop's route handler for path disclosure and FTP directory misconfigurations

What to Do	What You Should Observe
------------	-------------------------

2a. In ZAP Manual Request Editor, send GET requests to each path below. Note what each response returns. <pre>Target: http://localhost:3000 GET /rest/../../../../etc/passwd GET /api/../../../../package.json GET /ftp../package.json GET /assets../server.js GET /rest/this-route-does-not-exist</pre>	Key observations: /ftp/ directory: Juice Shop's /ftp/ route is a deliberate misconfiguration — it exposes files intended to be private. Traversal attempts against the REST API trigger 404 responses. Check whether the 404 body echoes the requested path back to the client. Path echo in 404: If the response body contains the string you submitted, this is path disclosure. Compare with <code>vulnerable_shop_fixed.py</code> which returns a generic <code>{"error": "Resource not found"}</code> with no path reflection. TRACE method (bonus): Change method to TRACE and resend any request. If Juice Shop reflects the full request back, this is Cross-Site Tracing (XST) — a lesser-known but exploitable misconfiguration.
--	--

Stage 3 — Boundary-Violating and Oversized Inputs

Test integer overflow, negative quantities, and oversized strings against Juice Shop APIs

What to Do	What You Should Observe
3a. Basket quantity manipulation Add any item to the Juice Shop basket via the UI. In ZAP History, find the <code>PUT /api/BasketItems/{id}</code> request. Send it to the Repeater. Modify the quantity field in turn: <pre>{"quantity": -999} {"quantity": 0} {"quantity": 9999999999999999} {"quantity": "DROP TABLE basket--"}</pre> Record whether each is accepted (200), rejected with a safe error (400), or crashes with a verbose error (500).	What to look for: Accepted negative quantity: Item price becomes negative — a direct financial impact vulnerability. This is A10 Mishandling of Exceptional Conditions: the application has no server-side guard for a value that is mathematically valid but semantically impossible. Integer overflow: Very large numbers may overflow JavaScript's Number type ($2^{53}-1$). A 500 error here discloses whether Juice Shop uses integer or float arithmetic internally. SQL string in quantity: A 400 with a safe message means input validation is working. A 500 with a Sequelize error means the string reached the ORM, confirming insufficient validation before database operations.
3b. Oversized registration input In ZAP Repeater, POST to <code>/api/Users/</code> with an oversized email field: <pre>{"email": "a".repeat(5000) + "@test.com", "password": "Test1234!", "passwordRepeat": "Test1234!", "securityQuestion": {"id":1}, "securityAnswer": "x"}</pre> <i>Note: construct the 5000-char string manually or in the browser console, then paste into ZAP.</i>	What to look for: Safe response (correctly handled): <pre>{"message": "Data too large for column"}</pre> Unsafe response (information disclosure): Any 500 error with a Sequelize <code>SequelizeDatabaseError</code>, column name, or database type string. The fix: a maximum length validator applied before the ORM is called, returning a safe 400 with a generic message. Length validation is trivial to implement and eliminates an entire class of disclosure.

Stage 4 — Malformed JWT Tokens on Authenticated Endpoints

Test Juice Shop's JWT error handling — does the response help an attacker calibrate their next attempt?

What to Do	What You Should Observe
4a. Log in to Juice Shop as any user. In ZAP History, find a <code>GET /rest/basket/{id}</code> request and send it to Repeater. 4b. Modify the Authorization header in turn: Test 1: completely invalid <pre>Authorization: Bearer not.a.real.jwt</pre>	Key observations: Test 1 — invalid token: Does the error message name the JWT library? Does it distinguish 'not a token' from 'bad signature'? Informative error messages help attackers understand the validation pipeline. Test 3 — bad signature: If the response differs from Test 1, the application is leaking information about how far token processing got before failing — a timing and content oracle.

Test 2: truncated payload Authorization: Bearer eyJ...[valid header].[cut] Test 3: valid header, bad sig Decode your JWT at jwt.io, change one character in the signature, re-encode Test 4: alg:none attack Change the header to {"alg":"none","typ":"JWT"}, remove the signature entirely, keep the dot	Test 4 — alg:none: This is the critical test. If Juice Shop accepts an unsigned token, any user can forge admin credentials. Modern JWT libraries reject alg:none by default — but the error response may still disclose the library version. Good error response: {"status": "error", "message": "Unauthorized"} — no library names, no token detail, same response for all JWT failure types.
---	---

Skill Track Challenges

EXPLORER	Complete Stages 1 and 2 only. For each ZAP response that returns a 500 error, identify and underline the specific phrase that discloses the framework name. Write one sentence explaining why a developer might think this error response is 'harmless'.
PRACTITIONER	Complete all four Stages. For each information disclosure you find, complete the following table entry: Endpoint Malformed Input HTTP Status Disclosed Info Attack It Enables Aim for at least six completed rows. Then, using vulnerable_shop_fixed.py as your reference, describe in two sentences the fix pattern that eliminates all six disclosures simultaneously.
EXPERT	Complete all four Stages. Then perform the alg:none JWT attack from Stage 4 — if it succeeds, use the forged admin token to call GET /api/Users/ and capture the full user list in ZAP. Finally, compare the information disclosed by Juice Shop's errors against what vulnerable_shop_fixed.py returns for the same inputs. Write a one-paragraph 'error handling policy' that a development team could adopt as a coding standard, referencing OWASP A10:2025.

Three-Way Comparison Exercise

Run the same malformed request against all three targets and record the response in each column. This comparison makes the fix concrete rather than theoretical.

Malformed Input	Juice Shop Response	vulnerable_shop.py	vulnerable_shop_fixed.py
{"email": null}			
GET /api/../package.json			
{"quantity": -999}			
Bearer not.a.real.jwt			
"email" = 5000 chars			

The A10 test: every unhandled exception that reaches the client is a gift to the attacker. A stack trace confirms the framework. A file path confirms the directory structure. A column name eliminates blind SQLi. The fix is not catching errors — it is catching errors correctly: log the full detail server-side, return nothing exploitable to the client, and respond with the same generic message for every failure type so errors cannot be used as oracles. OWASP A10:2025 — Mishandling of Exceptional Conditions. Flask reference: vulnerable_shop_fixed.py @app.errorhandler(500). Juice Shop equivalent challenge: "Error Handling" (2-star).
--